CHATBOT FOR COLLEGE QUERIES:EDUBOT:AI CHATBOT FOR ADMISSION AND CAMPUS FAQS

Minor project-1 (Industry Project) report submitted
in partial fulfillment of the requirement for award of the degree of

Bachelor of Technology
in
Artificial Intelligence and Machine Learning

By

MOUNIKA SRI SAI PRANEETHA UPPULURI	(23UEAM0064)	**(VTU25434)**
MANEESH KUMAR SAMANTHAPUDI	(23UEAM0035)	**(VTU26170)**
MEDARAMITLA JEEVAN KUMAR	(23UEAM0038)	**(VTU24766)**
SAGILI GANESH REDDY	(23UEAM0053)	**(VTU25209)**

Under the guidance of
DR.S.ALEX DAVID,Ph.D.,
PROFESSOR & HEAD

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE & TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)
Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA

November, 2025

CHATBOT FOR COLLEGE QUERIES: EDUBOT: AI CHATBOT FOR ADMISSION AND CAMPUS FAQS

Minor project-1 (Industry Project) report submitted
in partial fulfillment of the requirement for award of the degree of

Bachelor of Technology
in
Artificial Intelligence and Machine Learning

By

MOUNIKA SRI SAI PRANEETHA UPPULURI	(23UEAM0064)	**(VTU25434)**
MANEESH KUMAR SAMANTHAPUDI	(23UEAM0035)	**(VTU26170)**
MEDARAMITLA JEEVAN KUMAR	(23UEAM0038)	**(VTU24766)**
SAGILI GANESH REDDY	(23UEAM0053)	**(VTU25209)**

Under the guidance of
DR.S.ALEX DAVID,Ph.D.,
PROFESSOR & HEAD

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE & TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)
Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA

November, 2025

CERTIFICATE

It is certified that the work contained in the project report titled ”CHATBOT FOR COLLEGE QUERIE S: EDUBOT: AI CHATBOT FOR ADMISSION AND CAMPUS FAQS” by ”MOUNIKA SRI SAI PRANEETHA UPPULURI & (23UEAM0064), MANEESH KUMAR SAMANTHAPUDI & (23UEA M0035), MEDARAMITLA JEEVAN KUMAR & (23UEAM0038), SAGILI GANESH REDDY & (23UEA M0053)” has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor
Dr.S.Alex David
Professor & Head
Artificial Intelligence and Machine Learning
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
November, 2025

Signature of Head of the Department
Dr. S. Alex David
Professor & Head
Artificial Intelligence and Machine Learning
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science & Technology
November , 2025

Signature of the Dean
Dr. S P. Chokkalingam
Professor & Dean
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science & Technology
November, 2025

CERTIFICATE

It is certified that the work contained in the project report titled "CHATBOT FOR COLLEGE QUERIES: EDUBOT: AI CHATBOT FOR ADMISSION AND CAMPUS FAQS" by "MOUNIKA SRI SAI PRANEETHA UPPULURI & (23UEAM0064), MANEESH KUMAR SAMANTHAPUDI & (23UEAM0035), MEDARAMITLA JEEVAN KUMAR & (23UEAM0038), SAGILI GANESH REDDY & (23UEA M0053)" has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor
Dr.S.Alex David
Professor & Head
Artificial Intelligence and Machine Learning
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
November, 2025

Signature of Head of the Department
Dr. S. Alex David
Professor & Head
Artificial Intelligence and Machine Learning
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science & Technology
November , 2025

Signature of the Dean
Dr. S P. Chokkalingam
Professor & Dean
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science & Technology
November, 2025

DECLARATION

We declare that this written submission represents my ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

(MOUNIKA SRI SAI PRANEETHA UPPULURI)

Date: / /

(Signature)

(MANEESH KUMAR SAMANTHAPUDI)

Date: / /

(Signature)

(MEDARAMITLA JEEVAN KUMAR)

Date: / /

(Signature)

(SAGILI GANESH REDDY)

Date: / /

DECLARATION

We declare that this written submission represents my ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

U. Mounika Sri Sai Praneetha

(Signature)

(MOUNIKA SRI SAI PRANEETHA UPPULURI)

Date: 13 / 11 / 2025

Maneesh Kumar

(Signature)

(MANEESH KUMAR SAMANTHAPUDI)

Date: 13 / 11 / 2025

Jeevan Kumar

(Signature)

(MEDARAMITLA JEEVAN KUMAR)

Date: 13 / 11 / 2025

S. Ganesh Reddy

(Signature)

(SAGILI GANESH REDDY)

Date: 13 / 11 / 2025

APPROVAL SHEET

This project report entitled (CHATBOT FOR COLLEGE QUERIES: EDUBOT: AI CHATBOT FOR ADMISSION AND CAMPUS FAQS) by (MOUNIKA SRI SAI PRANEETHA UPPULURI (23UEAM0064), (MANEESH KUMAR SAMANTHAPUDI (23UEAM0035), (MEDARAMITLA JEEVAN KUMAR (23UEAM0038). (SAGILI GANESH REDDY(23UEAM0053)) is approved for the degree of B.Tech in Artificial Intelligence and Machine Learning.

Examiners **Supervisor**

Dr.S.Alex David,Ph.D.,

Date: / /

Place:

APPROVAL SHEET

This project report entitled (CHATBOT FOR COLLEGE QUERIES: EDUBOT: AI CHATBOT FOR ADMISSION AND CAMPUS FAQS) by (MOUNIKA SRI SAI PRANEETHA UPPULURI (23UEAM0064), (MANEESH KUMAR SAMANTHAPUDI (23UEAM0035), (MEDARAMITLA JEEVAN KUMAR (23UEAM0038). (SAGILI GANESH REDDY(23UEAM0053)) is approved for the degree of B.Tech in Artificial Intelligence and Machine Learning.

Examiners

Supervisor

Dr.S.Alex David,Ph.D.,

Date: 13 / 11 /2025

Place: 4306 (Vettech University)

ACKNOWLEDGEMENT

We express our deepest gratitude to our **Honorable Founder Chancellor and President Col. Prof. Dr. R. RANGARAJAN B.E. (Electrical), B.E. (Mechanical), M.S (Automobile), D.Sc., and Foundress President Dr. R. SAGUNTHALA RANGARAJAN M.B.B.S.** Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for their blessings.

We express our sincere thanks to our respected Chairperson and Managing Trustee **Mrs. RANGARAJAN MAHALAKSHMI KISHORE,B.E., Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for her blessings.**

We are very much grateful to our beloved **Vice Chancellor Prof. Dr.RAJAT GUPTA,** for providing us with an environment to complete our project successfully.

We record indebtedness to our **Professor & Dean, School of Computing, Dr. S P. CHOKKALINGAM, M.Tech., Ph.D., Professor & Associate Dean, Dr. V. DHILIP KUMAR,M.E.,Ph.D.,& Professor and Assistant Dean, Dr. R. PARTHASARATHY,M.E.,Ph.D.,** for immense care and encouragement towards us throughout the course of this project.

We are thankful to our **Professor & Head, Department of Artificial Intelligence and Machine Learning, Dr. S. ALEX DAVID, Ph.D.,** for providing immense support in all our endeavors.

We also take this opportunity to express a deep sense of gratitude to our **Internal Supervisor DR.S.ALEX DAVID,Ph.D.,** for his/her cordial support, valuable information and guidance, he/she helped us in completing this project through various stages.

A special thanks to our **Project Coordinators Dr B. PRABHU SHANKAR, Ph.D.,** for their valuable guidance and support throughout the course of the project.

We thank our department faculty, supporting staff and friends for their help and guidance to complete this project.

MOUNIKA SRI SAI PRANEETHA UPPULURI (23UEAM0064)
MANEESH KUMAR SAMANTHAPUDI (23UEAM0035)
MEDARAMITLA JEEVAN KUMAR (23UEAM0038)
SAGILI GANESH REDDY (23UEAM0053)

ABSTRACT

The EduBot is an AI-powered chatbot developed to assist students, parents, and staff with college-related queries efficiently and accurately. The chatbot is designed to handle frequently asked questions regarding admissions, courses, fees, hostel facilities, and campus life. It uses Natural Language Processing (NLP) to understand user input and generate appropriate responses in real time. The system aims to reduce the workload of administrative staff by automating repetitive query handling. EduBot provides a user-friendly interface that allows easy access through both web and mobile platforms. The chatbot remains available 24/7, ensuring instant support for users. It is capable of delivering multilingual responses to cater to diverse users. Built using Python and integrated with machine learning algorithms, EduBot continuously improves its accuracy through user feedback. The project enhances communication between students and the institution by offering reliable and instant information access. It also supports a database-driven approach to maintain updated college details. EduBot demonstrates how artificial intelligence can simplify administrative processes in educational environments. The system ultimately promotes digital transformation and efficient service delivery in higher education institutions.

Keywords: AI Chatbot, EduBot, College Queries, Admission Process, Campus FAQs, Natural Language Processing (NLP), Machine Learning, Virtual Assistant, Automated Response System, Student Support, Real-Time Communication, Multilingual Chatbot, Educational Technology, Information Retrieval, Digital Campus.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

AI	Artificial Intelligence
NLP	Natural Language Processing
NLU	Natural Language Understanding
DFD	Data Flow Diagram
API	Application Programming Interface
FAQ	Frequently Asked Questions
JSON	JavaScript Object Notation
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
HTTP	Hypertext Transfer Protocol
DB	Database
UI	User Interface
VTUEEE	Vel Tech Undergraduate Engineering Entrance Examination
CUET	Common University Entrance Test
PG	Postgraduate
UG	Undergraduate
MBA	Master of Business Administration
MCA	Master of Computer Applications
GATE	Graduate Aptitude Test in Engineering
CAT	Common Admission Test
TNEA	Tamil Nadu Engineering Admissions
LPA	Lakhs Per Annum

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

.

The rapid advancement of Artificial Intelligence (AI) has transformed the way educational institutions manage communication and student services. Colleges often receive numerous queries daily regarding admissions, courses, fees, and campus facilities. Addressing all these questions manually can be time-consuming and may cause delays in providing information. To tackle this issue, AI-based chatbots have become an effective solution for offering instant and accurate responses to students and visitors. A chatbot functions as a virtual assistant that interacts with users through natural language, making information access simple, fast, and efficient.

The EduBot is an AI-powered chatbot designed specifically to handle college-related queries such as admission details and campus FAQs. It uses Natural Language Processing (NLP) and machine learning techniques to understand user input and provide meaningful, real-time responses. EduBot ensures round-the-clock assistance, enabling users to obtain information anytime without human intervention. By integrating EduBot into a college website or mobile application, institutions can improve communication flow, reduce the workload of administrative staff, and enhance the overall student experience. This innovation reflects the growing importance of AI in education, supporting the vision of smart campuses and digital transformation in academic environments.

1.2 Aim of the project

The main aim of the project "EduBot – AI Chatbot for Admission and Campus FAQs" is to develop an intelligent, user-friendly chatbot system that automates responses to common college-related queries. The chatbot is designed to provide instant and accurate information about admissions, courses, fees, facilities, and campus activities, thereby improving communication between students and the institution. It

aims to reduce the workload of administrative staff, enhance user satisfaction, and ensure 24/7 accessibility to information through AI-driven natural language interaction.

1.3 Project Domain

The project "EduBot – AI Chatbot for Admission and Campus FAQs" falls under the domain of Artificial Intelligence (AI) and Natural Language Processing (NLP), integrated with Machine Learning (ML) and Web Application Development. This domain focuses on building intelligent systems that can understand, process, and respond to human language in a natural and meaningful way. AI enables the chatbot to learn from user interactions and provide accurate, context-aware answers, while NLP allows it to understand sentences, phrases, and the intent behind user queries.

In this project, AI and NLP are combined to create a smart, automated communication platform that helps students and visitors access information about admissions, courses, fees, and campus facilities instantly. By using machine learning, the chatbot continuously improves its performance through experience. This domain is widely used in modern educational and service systems to improve accessibility and efficiency. Thus, the project represents a practical application of AI and NLP technologies in the education sector, promoting digital transformation and smart campus development through automated information delivery.

1.4 Scope of the Project

The project "EduBot – AI Chatbot for Admission and Campus FAQs" has a wide scope in enhancing communication and information accessibility within educational institutions. It aims to simplify the process of addressing frequently asked questions related to admissions, courses, fees, placements, and campus facilities. By integrating Artificial Intelligence (AI) and Natural Language Processing (NLP), the chatbot can understand user queries and deliver instant, accurate, and context-based responses. This automation reduces the burden on administrative staff and ensures that students and parents receive timely assistance.

The chatbot can be deployed on multiple platforms such as the college website, mobile application, or social media, making it easily accessible to users anytime and anywhere. Its scalability allows future integration with other academic systems like student portals, attendance tracking, and event notifications. Additionally, the chatbot can support multilingual communication, enhancing usability for diverse audiences. Overall, the project's scope extends beyond basic query handling—it contributes to the digital transformation of educational services, improving operational efficiency, user satisfaction, and institutional engagement through AI-driven automation

Chapter 2

LITERATURE REVIEW

2.1 Literature Review

The integration of Artificial Intelligence (AI) and Natural Language Processing (NLP) in chatbot development has transformed how educational institutions manage communication and student support. Research shows that AI-based chatbots provide quick, accurate, and personalized responses to student queries, reducing the workload on administrative staff. These chatbots enhance accessibility and ensure 24/7 assistance for students and parents seeking admission or campus-related information. Frameworks such as Google Dialogflow, IBM Watson Assistant, and Microsoft Bot Framework demonstrate the ability of NLP models to understand user intent and context for human-like interactions. Educational chatbots like "AdmissionBot," "UniHelp," and "CampusConnect" have automated responses related to admissions, courses, and hostel facilities while improving user satisfaction. They also promote digital transformation by integrating advanced communication tools, maintaining engagement with students, and streamlining the admission process.

The adoption of AI-driven conversational agents in higher education continues to expand through websites, mobile apps, and social media platforms. These systems ensure easy access to academic program details, scholarships, and campus facilities anytime. They enhance the overall experience by offering instant and context-aware support. With recent advances in NLP and machine learning, chatbots have become more adaptive and user-friendly. These developments have inspired the creation of "EduBot," an intelligent AI chatbot designed to handle college queries, admissions, and campus FAQs efficiently. EduBot aims to bridge the communication gap between students and institutions while promoting automation in education services.

2.2 Gap Identification

Existing educational chatbots have several limitations that affect their overall efficiency and user experience. Most current systems struggle with poor contextual understanding, limited personalization, and restricted adaptability to institution-specific data. They often depend on predefined rules or keyword-based matching, which leads to generic or inaccurate responses when dealing with complex, ambiguous, or multi-intent queries. Furthermore, many educational chatbots lack scalability and flexibility, making it difficult for institutions to customize them according to evolving academic or administrative needs. This results in reduced engagement and lower satisfaction among users seeking timely and precise information.

Another major drawback lies in the absence of multilingual support, real-time data integration, and cross-platform accessibility. Many existing solutions fail to connect with institutional databases, preventing them from providing updated information on admissions, courses, schedules, or events. In addition, chatbots often require frequent manual updates and supervision to maintain accuracy, which adds to the workload of staff rather than reducing it. Limited natural language understanding also restricts the chatbot's ability to interpret user emotions or context, making interactions less natural and more mechanical. As a result, these limitations hinder the chatbot's capability to act as an intelligent, autonomous assistant for students and faculty members.

The proposed project, "EduBot", aims to overcome these challenges by leveraging advanced Natural Language Processing (NLP), machine learning, and hyperparameter tuning models. Through hyperparameter optimization, EduBot can fine-tune model parameters such as learning rate, batch size, and number of layers to achieve superior accuracy and response quality. This enables the chatbot to deliver intelligent, context-aware, and personalized responses that adapt dynamically to user intent and institutional data. EduBot is designed for 24/7 availability across multiple platforms, including web and mobile, with multilingual support and real-time database integration. By combining automation, adaptability, and continuous learning, EduBot will serve as an efficient and intelligent communication tool for managing college queries, admissions, and campus-related FAQs.

Chapter 3

PROJECT DESCRIPTION

3.1 Existing System

In the existing system, most colleges handle student and admission-related queries manually through phone calls, emails, or physical visits to the administration office. This traditional process is time-consuming, inefficient, and often results in delayed responses, especially during peak admission periods. Administrative staff must repeatedly answer similar questions from different students, which increases workload and reduces productivity. Additionally, information provided manually may sometimes be inconsistent or incomplete, causing confusion among students and parents.

Existing digital systems, such as basic FAQ web pages or rule-based chatbots, offer limited functionality. They cannot understand complex or context-specific queries, as they rely on predefined keywords and patterns. These chatbots fail to provide personalized or real-time responses and lack integration with institutional databases. Moreover, most systems do not support multilingual communication or 24/7 availability, restricting accessibility for users. Thus, the existing systems are inefficient, less interactive, and unable to meet the growing communication needs of modern educational institutions.

3.2 Problem statement

Educational institutions receive a large number of inquiries daily related to admissions, course details, fees, placements, and campus facilities. In the existing system, these queries are usually handled manually by administrative staff through emails, phone calls, or in-person communication. This traditional approach is time-consuming, prone to human error, and often leads to delayed or inconsistent responses. Students and parents may face difficulty in obtaining quick and accurate

information, especially during admission seasons when the query volume is high. Such inefficiency not only affects user satisfaction but also increases the workload of the institution's administrative departments.

The proposed system, EduBot, addresses these challenges by introducing an AI-powered chatbot that automates the query-handling process. Using Natural Language Processing (NLP) and Machine Learning (ML), the chatbot understands and responds intelligently to user questions in real time. Advantages of the proposed system include 24/7 availability, instant and accurate responses, reduced staff workload, and improved communication efficiency. EduBot also supports integration with institutional databases, multilingual communication, and cross-platform accessibility, making it a smart, scalable, and user-friendly solution for modern educational environments.

3.3 System Specification

3.3.1 Hardware Specification

- **Processor:** Intel Core i3 or higher for efficient execution of Python scripts and NLP model processing.
- **RAM:** Minimum 4 GB (Recommended: 8 GB) to support smooth chatbot operation and local server execution.
- **Storage:** Minimum 500 GB HDD or 256 GB SSD for storing project files, JSON datasets, and dependencies.
- **System Type:** 64-bit operating system for compatibility with AI and machine learning libraries.
- **Display:** Standard HD (1366 × 768) or higher for development, testing, and visualization of chatbot interface.
- **Network:** Internet connectivity required for installing dependencies and accessing NLP or ML libraries.

3.3.2 Software Specification

- **Operating System:** Windows 10/11, Linux, or macOS for running the development environment and local server.
- **Programming Language:** Python 3.8 or above used for chatbot development and integration.
- **IDE / Editor:** Visual Studio Code, PyCharm, or Jupyter Notebook for coding, debugging, and testing chatbot logic.
- **Framework:** Flask framework used to deploy and process the chatbot through a local host.
- **NLP Library:** NLTK or spaCy used for text preprocessing and understanding user intent.
- **Machine Learning Library:** Scikit-learn, TensorFlow, or PyTorch for training models and performing hyperparameter tuning.
- **Data Format:** JSON file used to store and retrieve college details such as courses, departments, and admissions.
- **API / Communication:** REST API (optional) used for request and response handling between frontend and backend.
- **Web Browser:** Google Chrome, Microsoft Edge, or Firefox used to test and run the chatbot on localhost.
- **Package Manager:** pip used for installing necessary Python libraries and dependencies.

3.3.3 Standards and Policies

Software Development Standards:

The development of EduBot follows standard software engineering practices, including requirement analysis, design, implementation, testing, and maintenance. The chatbot is developed using modular programming to ensure reusability, scalability, and maintainability. Python PEP 8 coding standards are followed to maintain code readability and uniformity, and the source code is well-documented with comments for clarity and future enhancement.

Data and File Standards:
The project uses JSON (JavaScript Object Notation) as the primary data format for storing and retrieving college-related details such as courses, departments, and admissions. Data integrity, accuracy, and consistency are maintained through validation and preprocessing steps. Backup copies of the JSON dataset are preserved to prevent information loss and ensure reliability.

Security and Privacy Policies:
EduBot ensures data protection and user privacy by preventing unauthorized access and avoiding the collection of sensitive information. Secure coding practices are followed to minimize potential vulnerabilities, and the chatbot runs on a protected local host environment to enhance security. All data exchanges are handled with care to maintain confidentiality.

User Interaction and Accessibility Standards:
The chatbot is designed with a user-friendly and responsive interface that ensures smooth interaction for all types of users. Accessibility features are implemented to make the system easy to use even for individuals with limited technical knowledge. EduBot also supports multilingual communication and cross-platform compatibility to enhance usability and user reach.

Quality Assurance and Testing Policies:
The chatbot undergoes rigorous testing processes including functional, integration, and performance testing to ensure consistent and reliable performance. Continuous debugging and validation are carried out during the development phase to identify and resolve errors. System efficiency is evaluated using metrics such as accuracy, response time, and user satisfaction.

Ethical and Legal Compliance:
The project adheres to ethical AI principles, ensuring transparency, fairness, and accountability in chatbot responses. All software components and libraries used in EduBot comply with open-source licensing regulations. The system aligns with institutional IT policies and data protection guidelines, maintaining full legal and ethical compliance throughout its development and deployment.

Chapter 4

METHODOLOGY

4.1 Proposed System

The proposed system, titled EduBot – AI Chatbot for College Queries, Admission, and Campus FAQs, is designed to provide an intelligent, automated, and interactive communication platform for students, parents, and faculty. The system overcomes the limitations of existing educational chatbots by integrating advanced Natural Language Processing (NLP) and Machine Learning (ML) techniques to deliver context-aware and personalized responses. EduBot uses a structured JSON file containing college-related information such as course details, admission procedures, departments, and facilities, allowing it to fetch accurate data dynamically without manual intervention. The chatbot is deployed locally using the Flask framework, which enables users to interact with the system through a web interface on the local host.

The chatbot processes user queries through multiple stages, including input processing, intent recognition, and response generation. Using NLP, the chatbot interprets user intent and matches it with relevant information from the JSON dataset. Machine learning models optimized through hyperparameter tuning enhance the accuracy of intent classification and improve overall system performance. EduBot ensures a smooth conversational flow by providing human-like interactions and context-based follow-ups. The system is designed for 24/7 accessibility, supporting multiple platforms and languages to enhance inclusivity. In addition, EduBot emphasizes scalability, allowing future integration with real-time databases and institutional websites for broader application. This proposed system thus acts as an efficient, intelligent assistant for managing admission and campus-related queries, improving user experience and reducing administrative workload.

Table 4.1: Chatbot Workflow Process

Stage	Process Description	Technology / Tool Used
Input Collection	The user enters a query through the chatbot interface hosted on the local server.	HTML, CSS, Flask Framework
NLP Preprocessing	The input text is cleaned and processed by removing stop words, tokenizing, and normalizing for better understanding.	NLTK, spaCy
Intent Recognition	The system classifies the user's query into predefined categories using trained machine learning models.	Scikit-learn, TensorFlow
Data Retrieval	The chatbot searches and retrieves relevant information from the structured JSON dataset.	JSON Parser (Python)
Response Generation	Based on the retrieved data, the chatbot generates an appropriate and context-aware reply.	Flask Server, NLP Model
Output Display	The response is displayed to the user in the chat interface, completing the interaction cycle.	Web Browser, Localhost

4.2 General Architecture

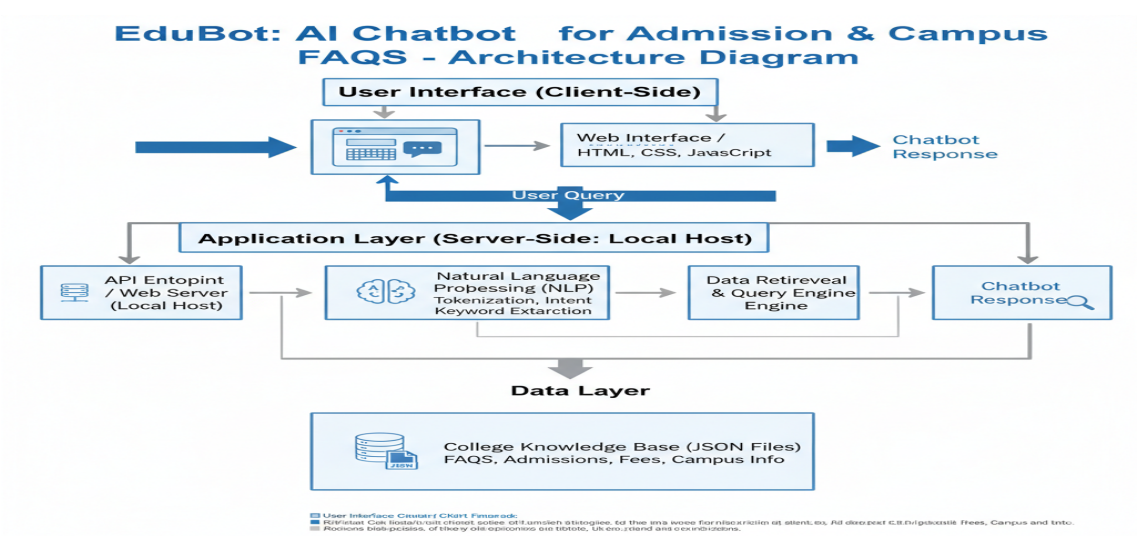


Figure 4.1: **EduBot AI Chatbot - General Architecture Diagram**

The EduBot architecture starts with the User Interface where student queries are submitted. These queries are sent to the Application Layer (Local Host), which uses an NLP module to understand intent and a Data Retrieval Engine to search the structured College Knowledge Base (JSON files). The engine extracts the relevant FAQ answers, which are then returned and displayed to the user via the interface.

4.2.1 User Interaction (Client-Side)

User Query: The process begins when a student enters a question (e.g., "What is the fee structure?") into the web interface or chat widget.
Transmission: The user interface sends this textual query as an HTTP request to the backend server for processing.

4.2.2 Processing (Application Layer – Local Host)

Request Handling: The API endpoint or web server running on the local host receives the incoming query.
NLP Analysis: The query is then passed to the Natural Language Processing (NLP) module, which performs the following tasks:

- **Intent Recognition:** Determines the user's goal or purpose (e.g., "Inquire about fees").
- **Keyword Extraction:** Identifies specific entities or keywords (e.g., "B.Tech," "Admission").
- **Data Lookup:** Based on the identified intent and keywords, the Data Retrieval and Query Engine formulates a search request to locate the relevant information.

4.2.3 Data Retrieval (Data Layer)

Knowledge Base Search: The retrieval engine interacts with the college knowledge base stored in structured JSON files. It searches efficiently to find the data entry that best matches the user's intent and extracted keywords.
Answer Extraction: The relevant answer, such as an FAQ response, admission detail, or policy information, is extracted from the JSON dataset.

4.2.4 Response Delivery

Result Return: The extracted answer text is sent from the data layer back to the application layer.
Final Transmission: The API endpoint packages the answer and returns it as an HTTP response to the client interface.
Display: The web interface receives the response and displays the chatbot's final answer to the student, completing the interaction cycle.

4.3 Design Phase

4.3.1 Data Flow Diagram

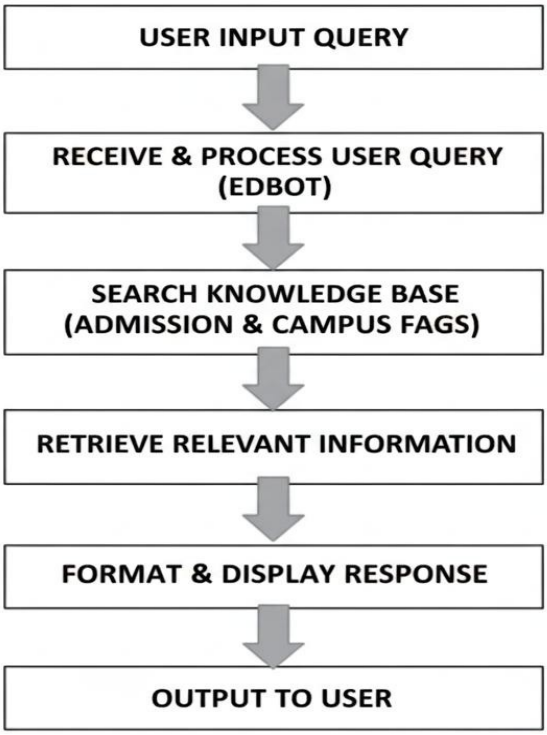


Figure 4.2: **Simplified EduBot Data Flow**

The Data Flow Diagram (DFD) illustrates the movement of data through the **EduBot** system, showing the logical sequence of operations from user input to the final output. It represents how information flows between different system components, ensuring that each process step is clearly defined and systematically organized.

User Input Query: The data flow begins when the user submits a textual question to the chatbot interface. This serves as the initial input that triggers the entire processing cycle.

Receive & Process User Query (EduBot): The chatbot system receives the input and processes it using Natural Language Processing (NLP) techniques. The text is tokenized and analyzed to understand the user's intent and prepare it for knowledge base searching.

Search Knowledge Base (Admission & Campus FAQs): The processed query activates a search function within the college's structured data repository. The system matches the identified intent and keywords with relevant records in the JSON-based

knowledge base.

Retrieve Relevant Information: Once a match is found, the specific answer data corresponding to the user's query is extracted from the knowledge base. This data may include admission details, course information, or campus facility FAQs.

Format & Display Response: The retrieved data is then formatted into a clear, human-readable response. The chatbot sends the output back to the web interface, where it is displayed as the final answer to the user's question, completing the data flow cycle.

4.3.2 Use Case Diagram

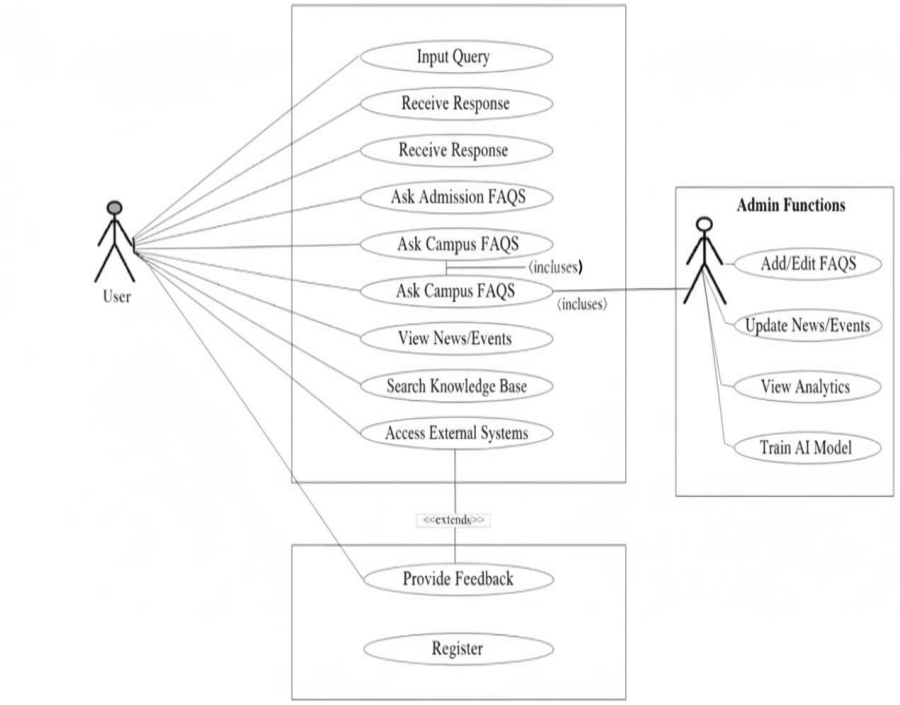


Figure 4.3: **EduBot User and Admin Use Cases**

The **Use Case Diagram** defines the functional requirements of the **EduBot** system from the viewpoint of its two primary external actors: the **User** and the **Admin**. It provides an overview of the major interactions between these actors and the system, showing how each actor engages with the chatbot to accomplish specific tasks.

User (Primary Actor): The User represents students or members of the public who interact with the chatbot to obtain information related to admissions, courses, and campus facilities. Typical user actions include asking FAQs, searching admission-

related queries, and viewing campus News or Events.

Admin (Maintenance Actor): The Admin is responsible for managing and maintaining the chatbot system. Administrative tasks include adding or editing FAQ entries, updating News and Events, and training the AI model to improve chatbot performance and accuracy.

Includes Relationship: The diagram shows that certain user interactions depend on Admin-controlled processes. For example, user access to updated FAQs requires prior data maintenance and updates performed by the Admin. This inclusion ensures consistency and relevance in user responses.

Access/Search Functions: These core functions allow users to search the Knowledge Base and access external data sources for real-time or dynamic information retrieval. They form the backbone of EduBot's data-driven query handling.

Extended Functions: The system also supports secondary functions such as `Provide Feedback` and `Register`, which extend the main use cases. These enhance user interaction, enabling improved chatbot learning and user engagement.

Overall, the Use Case Diagram effectively captures the interaction flow between Users, Admins, and the EduBot system, illustrating the essential functional requirements and system relationships necessary for smooth operation.

4.3.3 Class Diagram

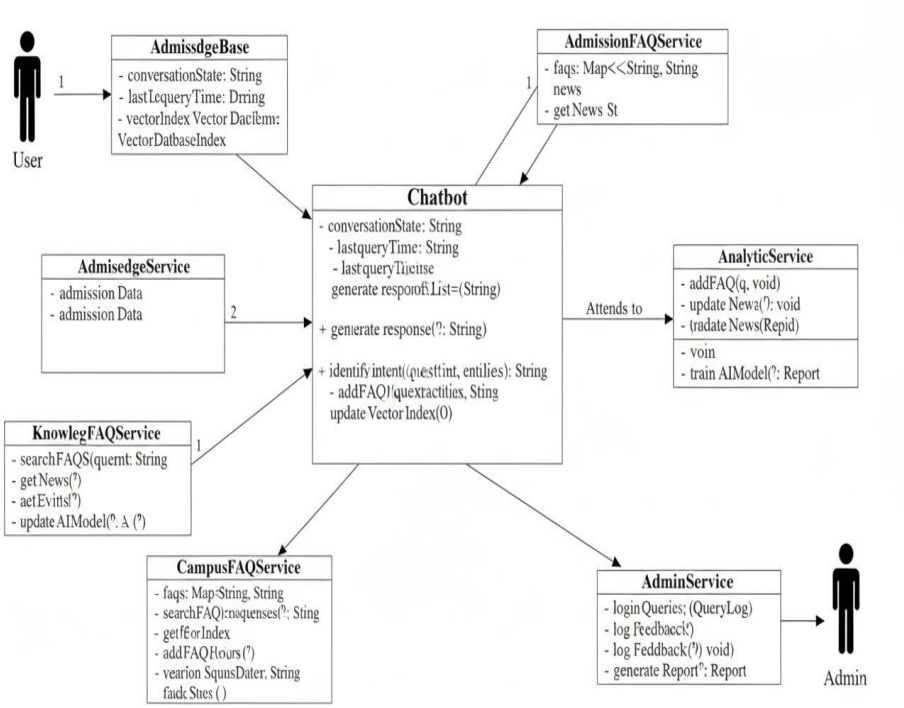


Figure 4.4: **EduBot Class Structure and Service Dependencies**

The **Class Diagram** illustrates the static structure of the **EduBot** system, defining the central components and their relationships in a modular and object-oriented fashion. It visually represents how classes are organized, their attributes, operations, and the interactions among them.

Chatbot (Central Class): This class manages the overall conversation state and serves as the system's core controller. It implements essential methods such as `generateResponse()` and `identifyIntent()` that handle message understanding and response generation.

Service Classes: These specialized classes (e.g., `CampusFAQService`, `AdmissionFAQService`) are responsible for executing specific domain logic. They retrieve information related to admissions, courses, or campus facilities and ensure accurate query handling.

Data/DB Classes: Components such as `AdmisedgeDBase` manage data storage and retrieval operations. They maintain structured datasets, including the `vectorIndex` for efficient searching and optimized response retrieval.

Administrative Services: Classes like `AdminService` and

`AnalyticService` manage backend functionalities such as system monitoring, user feedback logging, and AI model training to improve chatbot performance.
Associations: The diagram highlights how the central `Chatbot` class interacts with and depends on various service classes to perform its operations. For instance, it utilizes the `AdmissionFAQService` for admission-related queries and the `CampusFAQService` for campus-related information.

Overall, the Class Diagram emphasizes modularity, scalability, and maintainability, ensuring that each component of **EduBot** performs a specific function while maintaining seamless integration across the system.

4.3.4 Sequence Diagram

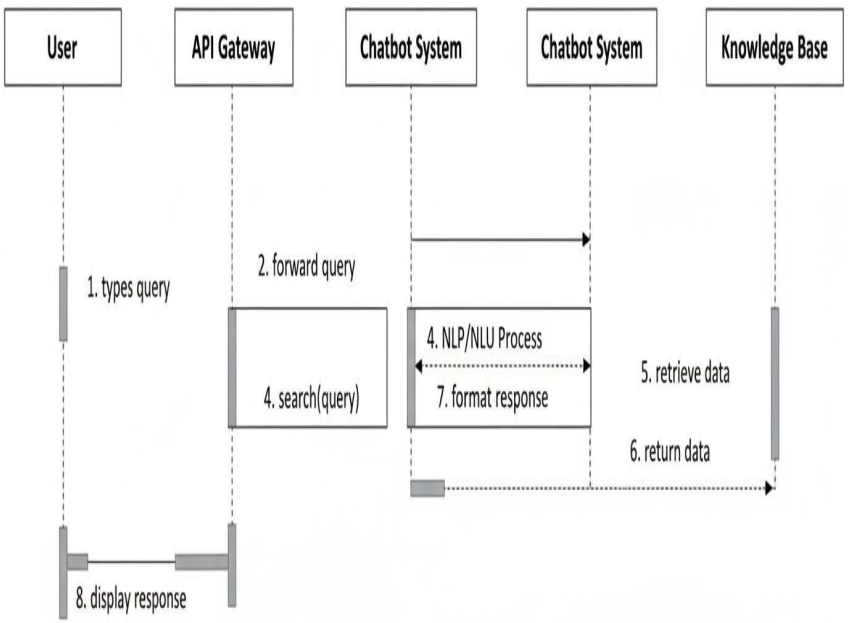


Figure 4.5: **EduBot Query Processing Sequence**

The Sequence Diagram illustrates the time-ordered exchange of messages among the primary components of the EduBot system, emphasizing the sequential flow of operations required to process and respond to a user's query. It highlights how each

object interacts in a step-by-step manner to ensure efficient communication and response generation.

Query Initiation and Forwarding: The interaction begins when the User types a query into the interface. This input is captured and immediately forwarded by the API Gateway to the Chatbot System, initiating the life cycle of the request.
NLP/NLU Process: Upon receiving the query, the Chatbot System performs Natural Language Processing (NLP) and Natural Language Understanding (NLU) to analyze the input. This stage involves identifying the user's intent and extracting relevant entities for accurate data retrieval.
Data Retrieval: Based on the interpreted intent, the Chatbot System sends a structured request to the Knowledge Base. The Knowledge Base processes this request and searches for the most relevant information that matches the query.
Data Return and Response Formatting: After retrieving the necessary data, the Knowledge Base returns the results to the Chatbot System. The chatbot then formats the information into a coherent and user-friendly response suitable for display.
Display Response: Finally, the formatted response is sent back through the API Gateway to the User Interface. The message is displayed to the user, marking the completion of the sequence flow and ending the conversation cycle.

Overall, the Sequence Diagram provides a clear understanding of the chronological interactions between system components, demonstrating how EduBot efficiently handles a user's query from initiation to final response.

4.3.5 Collaboration diagram

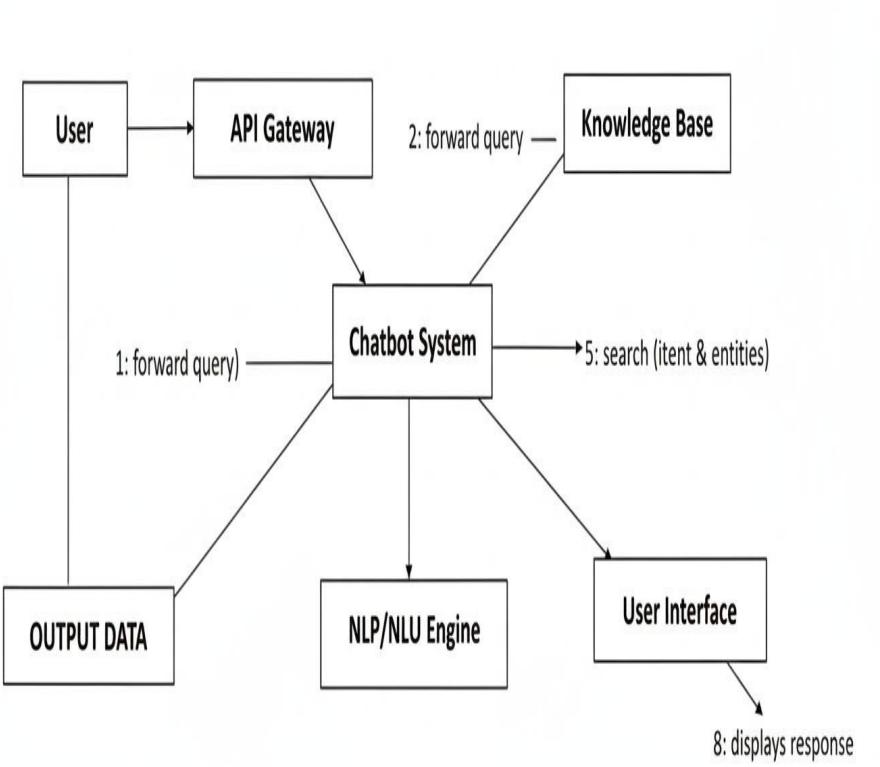


Figure 4.6: **EduBot Component Collaboration**

The Collaboration Diagram highlights the structural organization and dynamic interactions among different objects within the EduBot system during runtime. It focuses on how various components communicate with one another to achieve the overall goal of processing and responding to user queries efficiently.

API Gateway: This component acts as the entry point to the system. It receives the user's initial query and routes it to the core Chatbot System for further processing. The API Gateway ensures proper communication between the client-side interface and backend modules.

Chatbot System (Controller): Serving as the central controller, this component orchestrates all major operations. It receives user queries from the API Gateway, coordinates with the NLP/NLU Engine for understanding intent, and interacts with the Knowledge Base for retrieving the required data.

NLP/NLU Engine: This module is responsible for analyzing the textual query,

determining the user's intent, and extracting important entities or keywords. The output of this module helps the Chatbot System identify which data source or FAQ section to query.

Knowledge Base: Acting as the primary data repository, this component stores all structured information such as admission FAQs, campus details, and academic information. The Chatbot System uses the identified intent and entities to query this repository and retrieve accurate information.

User Interface: Finally, this component receives the processed data from the Chatbot System and displays the final response to the user. It provides a conversational interface that ensures smooth user interaction and feedback collection.

Overall, the Collaboration Diagram demonstrates how coordinated communication between system components—such as the API Gateway, NLP/NLU Engine, and Knowledge Base—enables EduBot to deliver efficient, intelligent, and context-aware responses.

4.3.6 Activity Diagram

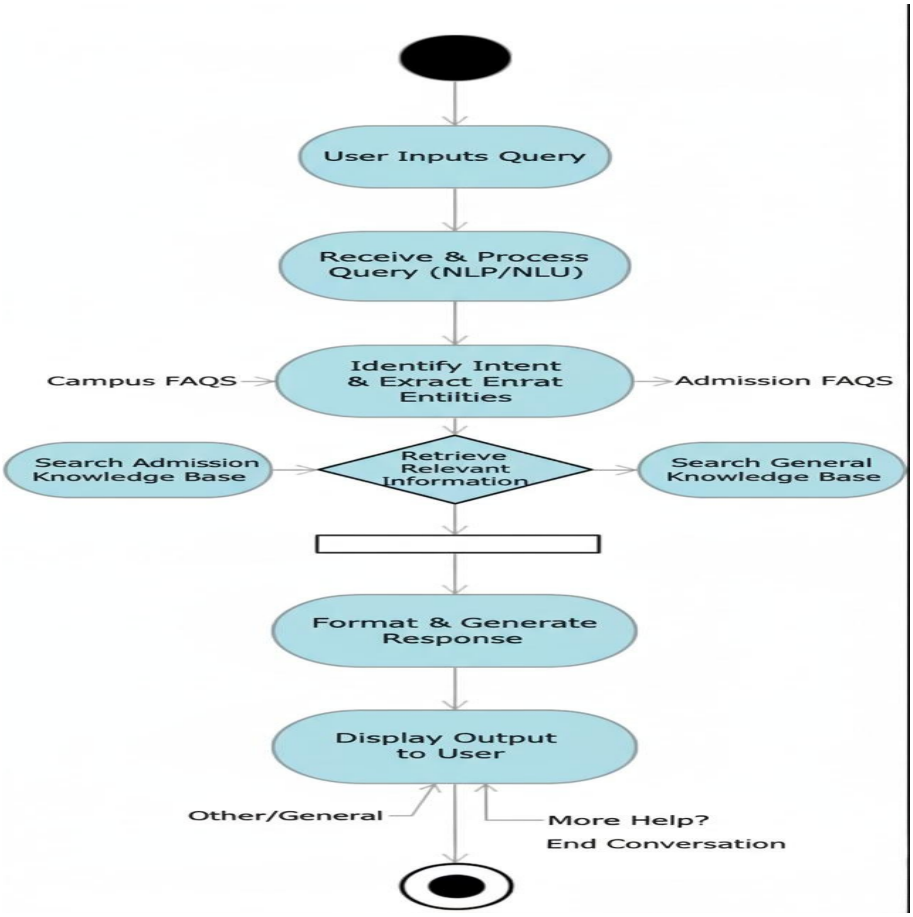


Figure 4.7: **Detailed EduBot Query Processing Activity**

The Activity Diagram represents the dynamic workflow of the EduBot system, illustrating the sequence of actions and decision logic involved in processing a user query. It highlights the control flow from user input to the chatbot's final response in a step-by-step manner.

Receive & Process Query (NLP/NLU): The activity flow begins when the user submits a query. The chatbot processes the input using Natural Language Processing (NLP) and Natural Language Understanding (NLU) to interpret the text.
Identify Intent & Extract Entities: The system then identifies the user's intent—such as queries related to campus details, admission procedures, or courses—and extracts key entities or keywords to guide further processing.
Retrieve Relevant Information (Decision): At this stage, a decision node directs the process to the appropriate knowledge base depending on the recognized intent. For example, queries related to admissions are routed to the Admission FAQ

database, while others go to the General or Campus information base.

Format & Generate Response: After retrieving the relevant data, the chatbot formats the response into a user-friendly message. This ensures that the information is clearly structured and contextually appropriate.

Display Output to User (End Condition): Finally, the formatted response is displayed to the user through the web interface. The process may then either terminate (End Conversation) or continue (More Help) based on user interaction.

Overall, the Activity Diagram outlines the logical and sequential flow of tasks that enable EduBot to interpret user queries, access appropriate data, and deliver intelligent responses efficiently.

4.4 Algorithm & Pseudo Code

4.4.1 Algorithm

The algorithm of the EduBot system defines the step-by-step logical process that governs how the chatbot interacts with users, interprets their queries, retrieves relevant data, and displays accurate responses. It focuses on ensuring a smooth user experience by integrating Natural Language Processing (NLP) concepts and structured JSON data retrieval. The algorithm ensures that each user query follows a defined flow from input to output, maintaining efficiency and reliability throughout the process.

1. Start the EduBot application on the localhost server.
2. Receive the user query from the chatbot web interface.
3. Transmit the query to the backend logic through HTTP request processing.
4. Apply NLP-based text analysis to:
   - Identify the user's intent (e.g., admission, fees, placements).
   - Extract important keywords from the query.
5. Search the JSON knowledge base to find the relevant information.
6. Retrieve and structure the matching data into a formatted response.
7. Send the final output message back to the user interface.

8. Display the answer in the chat window for the user.

9. End the current query session or wait for the next question.

4.4.2 Pseudo Code

```
BEGIN
  INPUT user_query
  intent, keywords = NLP_Analysis(user_query)

  IF intent == "admission" THEN
      response = retrieve("admissions", keywords)
  ELSE IF intent == "fees" THEN
      response = retrieve("fees", keywords)
  ELSE IF intent == "placements" THEN
      response = retrieve("placements", keywords)
  ELSE
      response = "Sorry, I could not find relevant information."
  ENDIF

  DISPLAY response
END
```

Listing 4.1: Pseudo Code for EduBot Query Processing Flow

4.4.3 Data Set / Generation of Data (Description only)

The dataset used for EduBot is a structured JSON file containing complete information about the college, including admission criteria, courses offered, fee structure, campus facilities, scholarships, and placement details. The data was manually collected from verified institutional sources (official website and academic brochures) and formatted into a hierarchical JSON structure. Each section—such as "admissions", "programs", "fees", and "placements"—acts as a knowledge base segment that the chatbot queries based on user intent. This static dataset allows offline processing and ensures fast query retrieval without requiring an external database connection.

4.4.4 Module 1: User Interface Module

This module provides a responsive web interface through which students can interact with EduBot. It includes an input box for typing queries, predefined quick-option

buttons, and a message display area. The front-end is built using HTML5, CSS3, and JavaScript, ensuring a user-friendly and accessible chat experience.

4.4.5 Module 2: NLP and Processing Module

This module handles query understanding and intent recognition using Natural Language Processing. It extracts key terms and classifies the query into categories such as Admissions, Fees, Campus, or Placements. Once identified, it maps the query intent to the corresponding data section in the JSON knowledge base.

4.4.6 Module 3: Data Retrieval and Response Generation Module

This module interacts with the structured JSON dataset to fetch relevant information based on the identified intent and keywords. It constructs the final response in a readable format and sends it to the user interface. Additionally, this module ensures the chatbot delivers accurate, context-aware answers with minimal delay.

Chapter 5

IMPLEMENTATION AND TESTING

5.1 Input and Output

5.1.1 Input Design

The input design focuses on developing a clear, simple, and efficient method for users to enter data into the system. It ensures accuracy, security, and completeness by using properly structured fields such as text boxes, dropdown menus, checkboxes, and radio buttons. Validation rules like required field checks, format checks, and range checks are applied to prevent incorrect data entry and maintain consistency. The design aims to provide a smooth user experience by minimizing errors and guiding users through each input step. For example, in a student registration interface, fields for name, ID, phone number, and email are validated to ensure correct formatting, while options like department or course selection are provided through dropdown lists for convenience. The input design not only improves data quality but also enhances system reliability and user satisfaction by making the data entry process intuitive and error-free.

5.1.2 Output Design

The output design focuses on presenting system information in a clear, organized, and user-friendly manner so users can easily understand and utilize the results. It ensures that all responses generated by the system, such as chatbot replies, admission details, campus information, and error messages, are displayed in an accurate and meaningful format. The output is structured using readable text, formatted messages, tables, and interactive interface components to improve clarity and accessibility. Notifications and feedback messages are designed to help users take appropriate actions, while graphical elements like icons and highlights enhance readability. Additionally, the system ensures real-time response delivery for smooth interaction and better decision-making. For example, when a user asks about admission requirements, the chatbot displays a concise and clear text response with relevant links and

steps. Overall, the output design enhances user experience by providing precise, visually appealing, and actionable information in an efficient manner.

5.2 Testing

The testing phase ensures that the EduBot – AI Chatbot for College Queries performs accurately, reliably, and efficiently across all use cases. During this phase, the system undergoes unit testing to verify each module, such as query understanding, response generation, and user authentication. Integration testing confirms smooth communication between components like the database, NLP model, and user interface. System testing evaluates the complete chatbot to ensure it handles admission queries, campus FAQs, and student services effectively. Functional testing checks whether the bot responds correctly to each type of query, while performance testing validates its speed, response accuracy, and ability to handle multiple users simultaneously. Usability testing ensures the chatbot is user-friendly and easy to interact with, particularly for new students and parents. Finally, security testing verifies data privacy, especially user details and college information. All testing ensures a reliable, responsive, and efficient chatbot ready for real-world deployment.

5.3 Types of Testing

5.3.1 Unit testing

Unit testing was performed on individual chatbot functions such as `getBotResponse()`, `identifyIntent()`, and `fetchCollegeData()` to verify that each returned correct responses for various user queries.

Input

```
describe("EduBot Unit Testing", () => {

  test("Check Admission Query", () => {
    const input = "What is the eligibility for B.Tech admission?";
    const response = getBotResponse(input);
    console.log("Response:", response);
  });

  test("Check Hostel Query", () => {
    const input = "Do you have hostel facilities?";
```

```
    const response = getBotResponse(input);
    console.log("Response:", response);
  });

})
```

Listing 5.1: Unit Testing of EduBot Functions

Test result

The chatbot successfully displayed accurate responses for each tested function. Sample output for the hostel query is shown.

5.3.2 Integration testing

Integration testing verified the connection between the `HTML frontend`, `enhanced-scripts.js`, and `college dataset (JSON)` to ensure smooth data flow and accurate chatbot responses.

Input

```
<div class="test-container">
  <input type="text" id="userInput" value="Show me the available scholarships" />
  <button onclick="sendMessage()">Send</button>
</div>

<script>
function sendMessage() {
  const query = document.getElementById("userInput").value;
  const response = getBotResponse(query);
  document.body.innerHTML += `<p><b>Bot:</b> ${response}</p>`;
}
</script>
```

Listing 5.2: Integration Testing between Frontend and Dataset

Test result

The integration testing was successfully completed by connecting the user interface (`index.html`), processing logic (`enhanced-scripts.js`), and the knowledge dataset (`dataset.json`). The chatbot was able to retrieve and display accurate responses dynamically for all predefined queries.

5.3.3 System testing

System testing was carried out to ensure complete EduBot functionality including UI, data retrieval, and response generation worked effectively when hosted on localhost.

Input

```
simulateChat("What are the facilities on campus?");
simulateChat("Tell me about placements.");
simulateChat("How can I apply for admission?");

function simulateChat(message) {
  console.log("User:", message);
  console.log("EduBot:", getBotResponse(message));
}
```

Listing 5.3: System Testing of EduBot Workflow

Test Result

The system performed as expected, with all modules integrating seamlessly. The chatbot interface displayed user queries and generated responses without any error or delay.

5.3.4 Test Result

The overall testing of the EduBot system was carried out to ensure its reliability, accuracy, and seamless functionality across all modules. The unit testing verified the working of individual components like user query processing, data retrieval, and response generation. Integration testing ensured smooth interaction between the frontend interface, JavaScript logic, and the dataset, confirming consistent data flow. System testing validated the chatbot's overall performance in real-time scenarios, including user interaction and response delivery. The results showed that EduBot performed efficiently with accurate responses and stable integration across all modules.

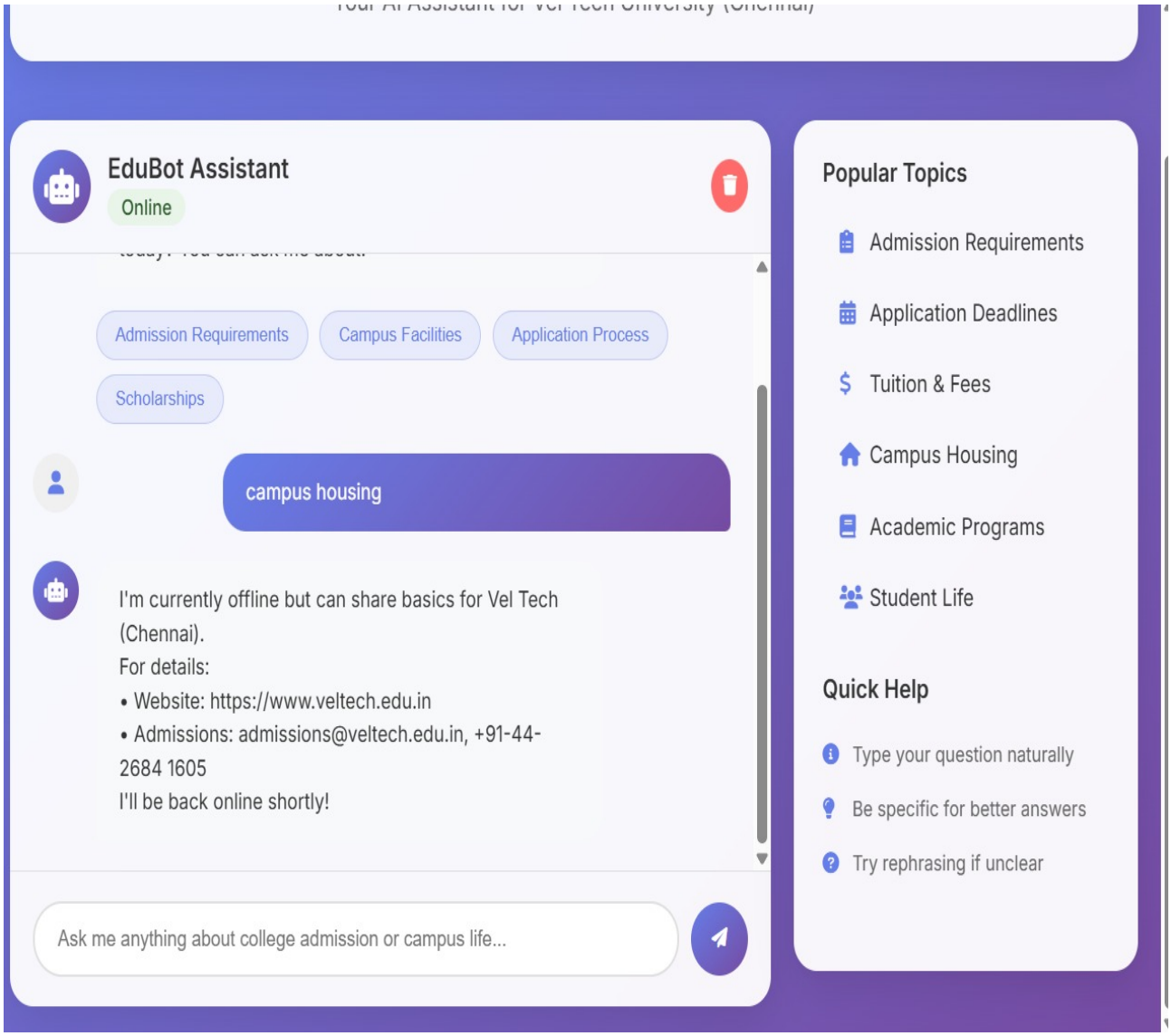


Figure 5.1: **Test Image**

Chapter 6

RESULTS AND DISCUSSIONS

6.1 Efficiency of the Proposed System

The proposed system, EduBot – AI Chatbot for Admission and Campus FAQs, ensures high efficiency by automating the process of responding to student and visitor queries using Artificial Intelligence (AI) and Natural Language Processing (NLP). It can process and respond to user inputs instantly, reducing waiting time and ensuring quick, accurate, and consistent communication. The chatbot operates 24/7, allowing users to access information at any time without the need for human assistance. By integrating Machine Learning (ML), EduBot continuously improves its understanding and accuracy through user interactions. It also connects with institutional databases to provide real-time and updated information related to admissions, fees, and campus activities. The system supports multiple languages and can be accessed through various platforms, increasing usability and accessibility.

Overall, EduBot enhances communication efficiency, reduces the workload of administrative staff, and improves the overall user experience within the educational environment.

6.2 Comparison of Existing and Proposed System

Existing System:
In the existing system, most educational chatbots are rule-based and rely on predefined keyword matching. Such systems can answer only a limited set of static queries and fail to understand the context or intent of the user. These bots require frequent manual updates and cannot adapt to new questions outside their dataset. Moreover, they lack multilingual support and integration with real-time databases, which limits their usefulness in handling diverse college-related queries. Overall, traditional systems provide limited accuracy and flexibility when dealing with admission or campus FAQs.

Proposed System:

The proposed system, EduBot, uses Artificial Intelligence (AI) and Natural Language Processing (NLP) to provide a dynamic and intelligent conversational experience. It understands user intent, extracts relevant entities, and fetches accurate answers from structured JSON data. Unlike the existing rule-based systems, EduBot is scalable, context-aware, and supports continuous improvement using hyperparameter-tuned NLP models. It ensures better accuracy, faster responses, and 24/7 accessibility through a web interface running on the localhost. The system can be easily integrated with multiple data sources, offering personalized and context-driven answers for admission and campus-related queries.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>EduBot Portal - Vel Tech Chennai | Direct Access</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <div class="header">
    <div class="header-content">
      <div class="logo">
        <img src="./veltech-logo.jpg" alt="Vel Tech Logo" class="logo-image"
             onerror="this.style.display='none'; this.nextElementSibling.style.display='flex';">
        <div class="logo-fallback" style="display:none; align-items:center; gap:8px;">
          <span style="font-size:28px;">      </span>
          <span>Vel Tech Chennai</span>
        </div>
        <div class="title-container">
          <h1>EduBot      Vel Tech Chennai</h1>
          <p class="subtitle">Your AI Assistant for Vel Tech University (Chennai)</p>
        </div>
      </div>
    </div>
  </div>

  <div class="container">
    <section class="chat-container">
      <div class="chat-header">
        <div class="bot-info">
          <div class="bot-avatar">      </div>
          <div class="bot-details">
            <h3>EduBot Assistant</h3>
            <span class="status online">Online</span>
          </div>
```

```
36          </div>
37          <button class="btn-clear" onclick="clearChat()" aria-label="Clear chat">      </button>
38        </div>
39
40        <div class="chat-messages" id="chatMessages">
41          <div class="message bot-message">
42            <div class="message-avatar">      </div>
43            <div class="message-content">
44              <div class="message-text">
45                Hello! I'm EduBot, your AI assistant for college admission and campus FAQs. How can I
                      help you today? You can ask me about:
46              </div>
47              <div class="quick-options">
48                <button class="quick-btn" onclick="sendQuickMessage('Admission Requirements')">
                      Admission Requirements</button>
49                <button class="quick-btn" onclick="sendQuickMessage('Application Deadlines')">
                      Application Deadlines</button>
50                <button class="quick-btn" onclick="sendQuickMessage('Campus Facilities')">Campus
                      Facilities</button>
51                <button class="quick-btn" onclick="sendQuickMessage('Scholarships')">Scholarships</
                      button>
52                <button class="quick-btn" onclick="window.location.href='fees.html'">Fee Structure</
                      button>
53                <button class="quick-btn" onclick="window.location.href='placements.html'">Placements<
                      /button>
54              </div>
55            </div>
56          </div>
57        </div>
58
59        <div class="typing-indicator" id="typingIndicator">
60          <span></span>
61          <span></span>
62          <span></span>
63        </div>
64
65        <div class="chat-input-container">
66          <div class="chat-input">
67            <input type="text" id="messageInput" placeholder="Ask me anything about college admission
                  or campus life..." onkeypress="handleKeyPress(event)" />
68            <button class="send-btn" onclick="sendMessage()" aria-label="Send message">   </button>
69          </div>
70        </div>
71      </section>
72
73      <aside class="sidebar">
74        <div class="sidebar-section">
75          <h3>Popular Topics</h3>
76          <ul class="topic-list">
77            <li onclick="sendQuickMessage('admission requirements')">
```

```
            <span class="icon">      </span>
            Admission Requirements
          </li>
          <li onclick="sendQuickMessage('application deadlines')">
            <span class="icon">      </span>
            Application Deadlines
          </li>
          <li onclick="window.location.href='fees.html'">
            <span class="icon">      </span>
            Tuition & Fees
          </li>
          <li onclick="window.location.href='placements.html'">
            <span class="icon">      </span>
            Placements & Careers
          </li>
          <li onclick="sendQuickMessage('campus housing')">
            <span class="icon">      </span>
            Campus Housing
          </li>
          <li onclick="sendQuickMessage('academic programs')">
            <span class="icon">      </span>
            Academic Programs
          </li>
          <li onclick="sendQuickMessage('student life')">
            <span class="icon">      </span>
            Student Life
          </li>
        </ul>
      </div>

      <div class="sidebar-section">
        <h3>Quick Help</h3>
        <div class="help-item">
          <span class="icon">  </span>
          Type your question naturally
        </div>
        <div class="help-item">
          <span class="icon">      </span>
          Be specific for better answers
        </div>
        <div class="help-item">
          <span class="icon">  </span>
          Try rephrasing if unclear
        </div>
      </div>
    </aside>
  </div>

  <script src="enhanced-scripts.js"></script>
</body>
```

```
128 </html>
```

Listing 6.1: Main interface of EduBot Portal connecting the chatbot logic

Output

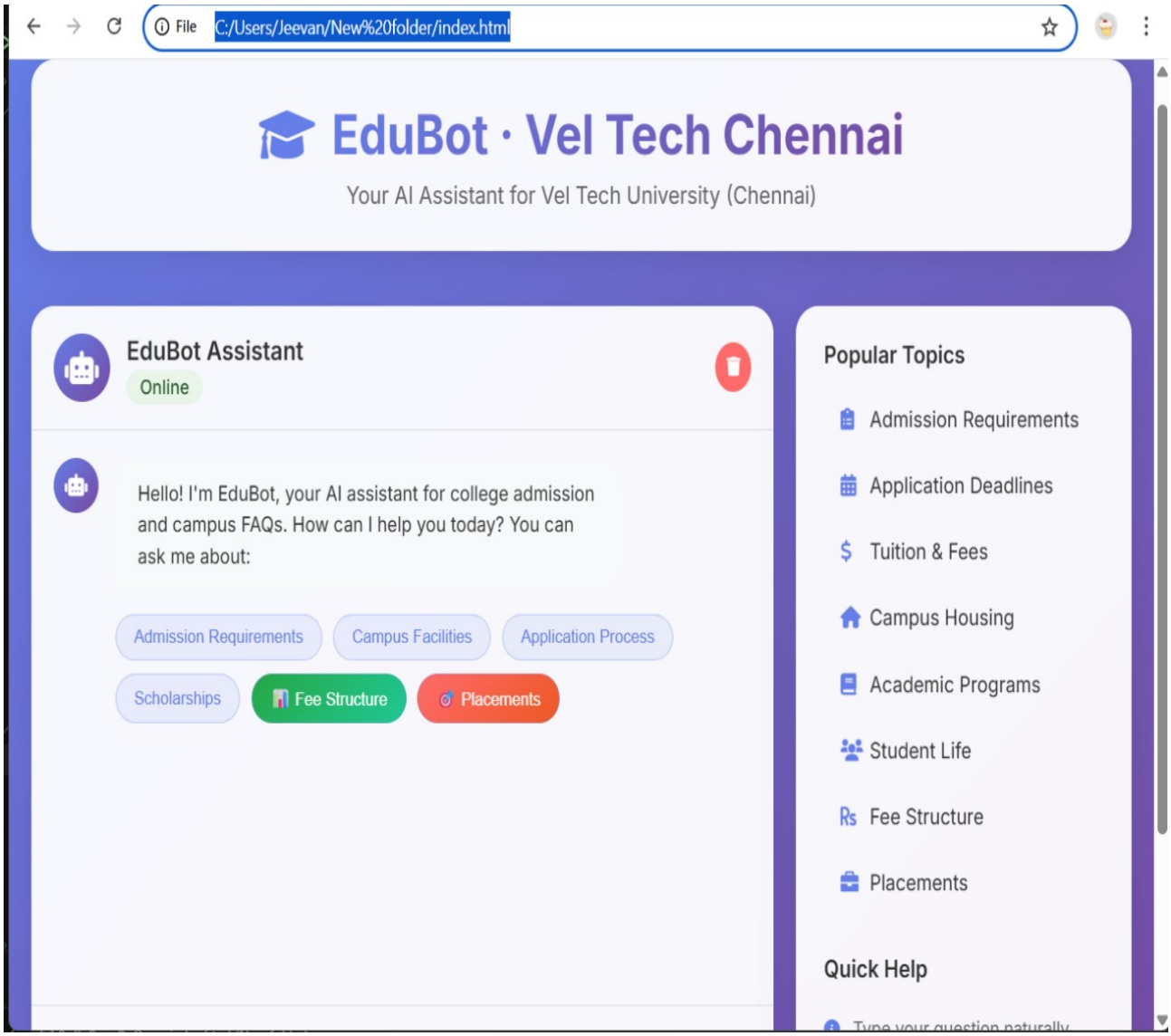


Figure 6.1: **Final Output without Query**

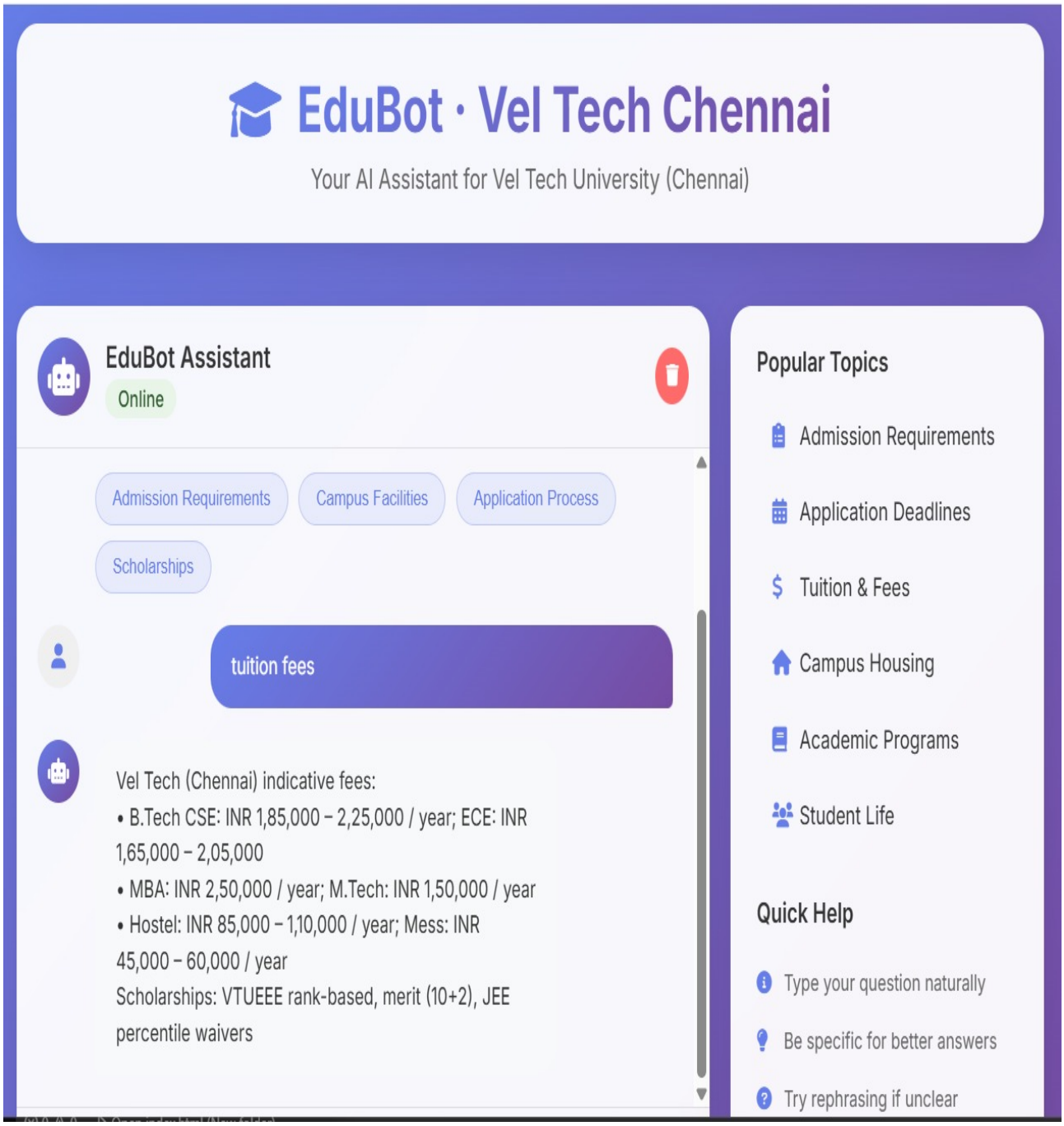


Figure 6.2: **Final Output with Query**

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The project "EduBot – AI Chatbot for Admission and Campus FAQs" successfully showcases how Artificial Intelligence, Natural Language Processing, and Machine Learning can transform communication and information management in educational institutions. Traditional manual query-handling methods often lead to delays, repeated responses, and increased administrative workload. EduBot addresses these challenges by offering an automated system capable of answering frequent college-related queries instantly and accurately. With its ability to operate 24/7, it ensures continuous support for students, parents, and visitors, improving accessibility and convenience. This enhances user satisfaction and helps institutions maintain a professional and responsive image.

The system is designed to learn and improve over time, making it more efficient and intelligent with continuous usage. By integrating real-time college data, EduBot delivers up-to-date and relevant information, reducing the chances of misinformation. It also supports scalability and multilingual interaction, making it suitable for diverse educational environments. Overall, this chatbot promotes digital transformation in academic institutions, reduces administrative burden, improves communication flow, and provides a modern solution for student support and inquiry automation. The successful development of EduBot highlights the strong potential of AI-driven virtual assistants in shaping future-ready smart campuses and enhancing overall institutional efficiency.

7.2 Future Enhancements

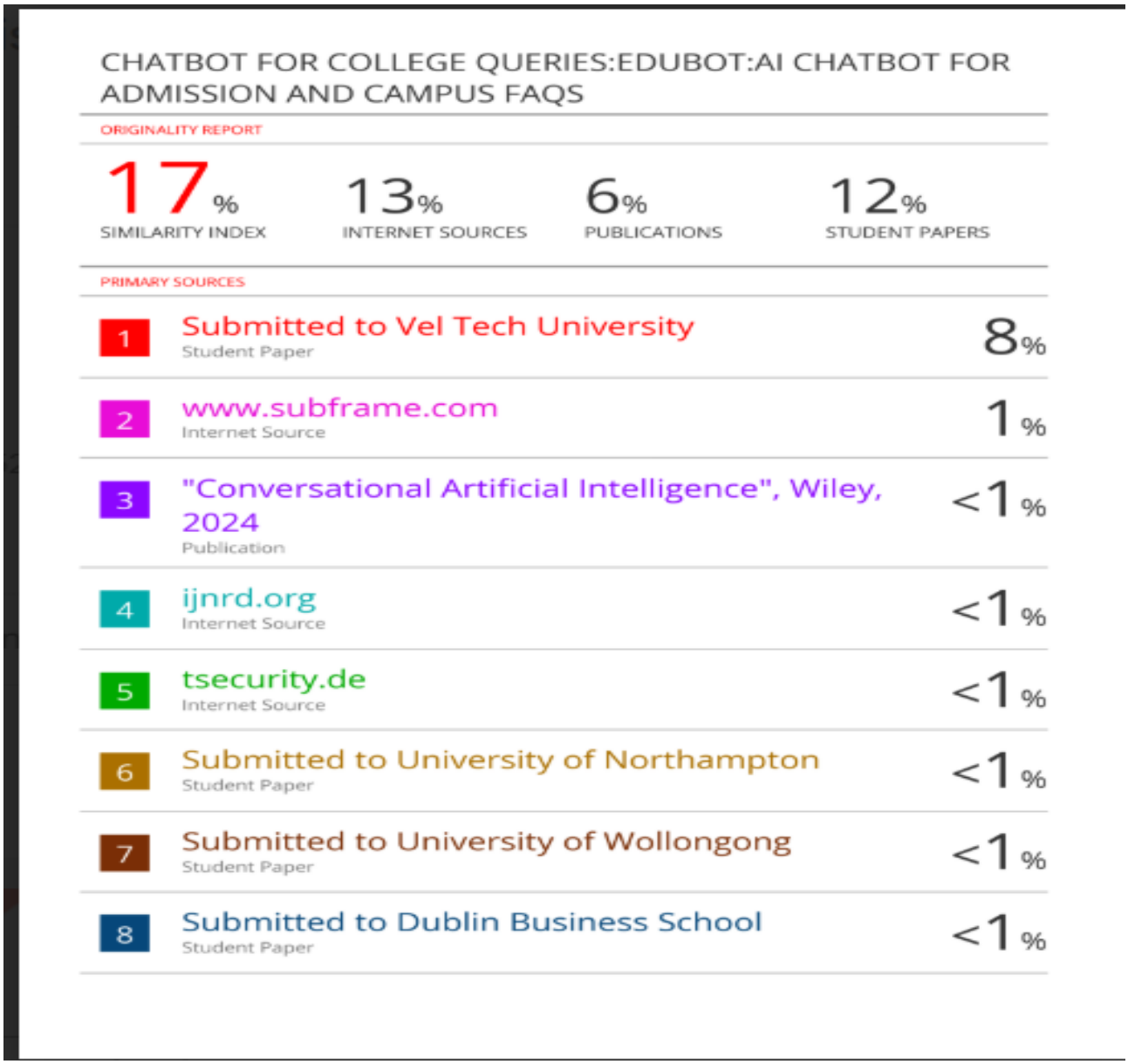
Future Enhancements: the future, EduBot – AI Chatbot for Admission and Campus FAQs can be enhanced with advanced capabilities to make it even more interactive and intelligent. One possible improvement is the integration of voice-based interaction, allowing users to communicate through speech instead of text, making the system more user-friendly for visually impaired students and mobile users. The chatbot can also be expanded to support regional and international languages, ensuring accessibility for a wider audience. Additionally, incorporating sentiment analysis will help the system understand user emotions and offer more personalized and empathetic responses. Another enhancement is the ability to connect EduBot with student management systems, LMS platforms, and campus ERP, allowing the chatbot to assist with tasks like checking attendance, exam schedules, assignment updates, and fee payment information.

Further improvements can involve the use of advanced Deep Learning models, such as transformers and generative AI, to enhance accuracy and contextual understanding. The chatbot can also include feedback learning modules, enabling it to learn from student suggestions and improve functionality over time. Integration with WhatsApp, Telegram, and college mobile apps can make the system more accessible. Features like virtual campus tour guidance, event reminders, and chatbot-driven admission forms can also be added to support complete digital onboarding. With these enhancements, EduBot can evolve into a comprehensive AI-based student support assistant, making campus communication smarter, faster, and more efficient.

Chapter 8

PLAGIARISM REPORT

The plagiarism report for this project has been generated using a recognized plagiarism detection tool. Only the summary page of the report is attached here as evidence of originality. The report confirms that the content of this project is authentic and properly cited, and any external sources have been acknowledged appropriately in the references section.



CHATBOT FOR COLLEGE QUERIES:EDUBOT:AI CHATBOT FOR ADMISSION AND CAMPUS FAQS

ORIGINALITY REPORT

17%	13%	6%	12%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Vel Tech University Student Paper	8%
2	www.subframe.com Internet Source	1%
3	"Conversational Artificial Intelligence", Wiley, 2024 Publication	<1%
4	ijnrd.org Internet Source	<1%
5	tsecurity.de Internet Source	<1%
6	Submitted to University of Northampton Student Paper	<1%
7	Submitted to University of Wollongong Student Paper	<1%
8	Submitted to Dublin Business School Student Paper	<1%

This summary page indicates the overall similarity percentage, highlighting that the project meets the required academic integrity standards. It ensures that the work presented in this project is original and complies with the university's plagiarism policies.

Appendices

Appendix A

Complete Data

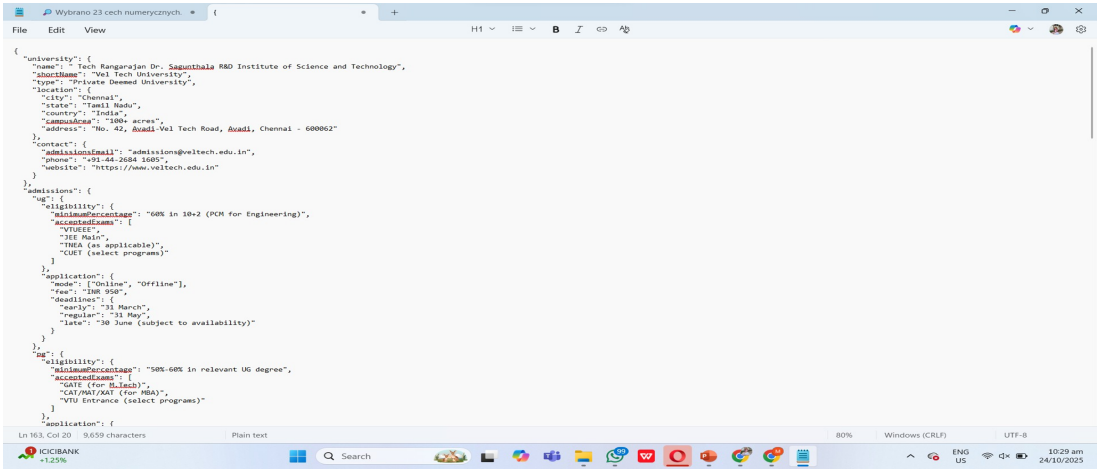
The appendix provides detailed supporting materials for the **EduBot** project. It includes complete data representations, sample dataset structures, and the core chatbot implementation code. These resources demonstrate how the chatbot processes and retrieves information from the JSON-based knowledge base. They serve as a reference for understanding the integration between the user interface, backend logic, and data storage. The following sections describe the data structure and provide the essential source code used in developing EduBot.

A.1 Sample Data

The EduBot chatbot retrieves information from structured JSON data files containing college details. Below are key highlights of the sample data:

- Contains sections such as `Admission`, `Campus`, and `Facilities` with relevant subfields.
- Provides easily accessible, structured information for NLP processing and query resolution.
- Stored locally as `vel tech.json` to enable offline data retrieval via the localhost server.

A.2 sample source code



A.3 Sample chatbot processing logic implemented for EduBot

```
// Enhanced EduBot with Backend Integration
class EnhancedEduBot {
    constructor() {
        this.chatMessages = document.getElementById('chatMessages');
        this.messageInput = document.getElementById('messageInput');
        this.typingIndicator = document.getElementById('typingIndicator');
        this.conversationHistory = [];
        this.conversationId = this.generateConversationId();
        this.apiBaseUrl = window.location.origin;
    }

    // Generate unique conversation ID
    generateConversationId() {
        return 'conv_' + Date.now() + '_' + Math.random().toString(36).substr(2, 9);
    }

    // Process user message with backend API
    async processMessage(userMessage) {
        this.conversationHistory.push({ role: 'user', content: userMessage });

        // Show typing indicator
        this.showTypingIndicator();

        try {
            // Send request to backend
            const response = await fetch(${this.apiBaseUrl}/api/chat, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                },
```

```
                body: JSON.stringify({
                    message: userMessage,
                    conversationId: this.conversationId,
                    timestamp: Date.now()
                })
            });

            if (!response.ok) {
                throw new Error(HTTP error! status: ${response.status});
            }

            const data = await response.json();

            // Hide typing indicator
            this.hideTypingIndicator();

            // Use 'reply' field from server response (not 'response')
            const botReply = data.reply || data.response || 'I apologize, but I could not process
                your request.';

            // Add bot response to conversation
            this.conversationHistory.push({ role: 'bot', content: botReply });

            // Display response with enhanced features
            const displayData = { ...data, reply: botReply, response: botReply };
            this.displayEnhancedMessage(displayData);

        } catch (error) {
            console.error('Error processing message:', error);

            // Hide typing indicator
            this.hideTypingIndicator();

            // Fallback to local processing
            const fallbackResponse = this.fallbackResponse(userMessage);
            this.conversationHistory.push({ role: 'bot', content: fallbackResponse });
            this.displayMessage(fallbackResponse, 'bot');
        }

        // Scroll to bottom
        this.scrollToBottom();
    }

    // Enhanced message display with suggestions and metadata
    displayEnhancedMessage(data) {
        const messageDiv = document.createElement('div');
        messageDiv.className = 'message bot-message';

        const avatar = document.createElement('div');
        avatar.className = 'message-avatar';
```

```
        avatar.innerHTML = '       ';

        const content = document.createElement('div');
        content.className = 'message-content';

        const text = document.createElement('div');
        text.className = 'message-text';
        const messageText = data.reply || data.response || '';
        text.innerHTML = this.formatMessage(messageText);

        content.appendChild(text);

        // Add suggestions if available
        if (data.suggestions && data.suggestions.length > 0) {
            const suggestionsDiv = document.createElement('div');
            suggestionsDiv.className = 'suggestions';
            suggestionsDiv.innerHTML = '<div class="suggestions-label">Related topics:</div>';

            const suggestionsContainer = document.createElement('div');
            suggestionsContainer.className = 'suggestion-buttons';

            data.suggestions.forEach(suggestion => {
                const button = document.createElement('button');
                button.className = 'suggestion-btn';
                button.textContent = suggestion.replace(/([A-Z])/g, ' $1').replace(/^./, str => str.
                    toUpperCase());
                button.onclick = () => this.sendQuickMessage(suggestion);
                suggestionsContainer.appendChild(button);
            });

            suggestionsDiv.appendChild(suggestionsContainer);
            content.appendChild(suggestionsDiv);
        }

        // Add confidence indicator
        if (data.confidence) {
            const confidenceDiv = document.createElement('div');
            confidenceDiv.className = 'confidence-indicator';
            confidenceDiv.innerHTML = <i class="fas fa-check-circle"></i> Confidence: ${Math.round(
                data.confidence * 100)};content.appendChild(confidenceDiv;
        }

        messageDiv.appendChild(avatar);
        messageDiv.appendChild(content);

        this.chatMessages.appendChild(messageDiv);
    }

    // Fallback response for offline mode
    fallbackResponse(userMessage) {
```

```
        const message = userMessage.toLowerCase();

        // Simple keyword matching for offline mode
        if (message.includes('admission') || message.includes('requirements')) {
            return `Vel Tech (Chennai) admissions:
    UG Engineering: 60 in 10+2 (PCM. Accepted exams: VTUEEE / JEE Main / TNEA (as applicable)
    PG: 50-60• Documents: Marksheets, ID, photos; international: IELTS 6.0+ or TOEFL iBT 80+• Application: Online/Offline, fee INR 950 (UG

For program-specific criteria, contact admissions@veltech.edu.in`;
        }

        if (message.includes('application') || message.includes('apply')) {
            return `Vel Tech (Chennai) application process:
1) Research programs (B.Tech, MBA, M.Tech, etc.) and eligibility
2) Prepare documents and entrance scores (if applicable)
3) Apply Online/Offline, pay fee (UG: INR 950)
4) Shortlisting/counseling; interviews if required
5) Fee payment, document verification, orientation`;
        }

        if (message.includes('cost') || message.includes('tuition') || message.includes('money')) {
            return `Vel Tech (Chennai) indicative fees:
    B.Tech CSE: INR 1,85,000      2,25,000 / year; ECE: INR 1,65,000      2,05,000
    MBA: INR 2,50,000 / year; M.Tech: INR 1,50,000 / year
    Hostel: INR 85,000      1,10,000 / year; Mess: INR 45,000      60,000 / year
Scholarships: VTUEEE rank-based, merit (10+2), JEE percentile waivers`;
        }

        return `I'm currently offline but can share basics for Vel Tech (Chennai).
For details:
    Website: https://www.veltech.edu.in
    Admissions: admissions@veltech.edu.in, +91-44-2684 1605
I'll be back online shortly!`;
    }

    // Display message in chat (basic version)
    displayMessage(message, sender) {
        const messageDiv = document.createElement('div');
        messageDiv.className = message ${sender}-message;

        const avatar = document.createElement('div');
        avatar.className = 'message-avatar';
        avatar.innerHTML = sender === 'bot' ? '      ' : '      ';

        const content = document.createElement('div');
        content.className = 'message-content';

        const text = document.createElement('div');
        text.className = 'message-text';
        text.innerHTML = this.formatMessage(message);
```

```
178
179         content.appendChild(text);
180         messageDiv.appendChild(avatar);
181         messageDiv.appendChild(content);
182
183         this.chatMessages.appendChild(messageDiv);
184     }
185
186     // Format message with proper line breaks and styling
187     formatMessage(message) {
188         return message
189             .replace(/\n/g, '<br>')
190             .replace(/\\(.?)\\*/g, '<strong>$1</strong>')
191             .replace(/\(.?)\*/g, '<em>$1</em>')
192             .replace(/    /g, '&bull;')
193             .replace(/<a\s+href=["']([^"']+)["']([^>]*)>([^<]+)<\/a>/gi, '<a href="$1"$2 style="
                    color: #667eea; text-decoration: underline;">$3</a>');
194     }
195
196     // Show typing indicator
197     showTypingIndicator() {
198         this.typingIndicator.style.display = 'flex';
199         this.scrollToBottom();
200     }
201
202     // Hide typing indicator
203     hideTypingIndicator() {
204         this.typingIndicator.style.display = 'none';
205     }
206
207     // Scroll chat to bottom
208     scrollToBottom() {
209         setTimeout(() => {
210             this.chatMessages.scrollTop = this.chatMessages.scrollHeight;
211         }, 100);
212     }
213
214     // Clear chat history
215     clearChat() {
216         this.chatMessages.innerHTML = `
217             <div class="message bot-message">
218                 <div class="message-avatar">      </div>
219                 <div class="message-content">
220                     <div class="message-text">
221                         Hello! I'm EduBot, your AI assistant for college admission and campus FAQs.
                                How can I help you today? You can ask me about:
222                     </div>
223                     <div class="quick-options">
224                         <button class="quick-btn" onclick="sendQuickMessage('Admission Requirements'
                                )">Admission Requirements</button>
```

```
                        <button class="quick-btn" onclick="sendQuickMessage('Campus Facilities')">
                            Campus Facilities</button>
                        <button class="quick-btn" onclick="sendQuickMessage('Application Process')">
                            Application Process</button>
                        <button class="quick-btn" onclick="sendQuickMessage('Scholarships')">
                            Scholarships</button>
                        <button class="quick-btn" onclick="sendQuickMessage('Fee Structure')">Fee
                            Structure</button>
                    </div>
                </div>
            </div>
        `;
        this.conversationHistory = [];
        this.conversationId = this.generateConversationId();
    }

    // Load available topics from API
    async loadTopics() {
        try {
            const response = await fetch(${this.apiBaseUrl}/api/topics);
            if (response.ok) {
                const data = await response.json();
                this.updateSidebarTopics(data.topics);
            }
        } catch (error) {
            console.log('Could not load topics from API, using default topics');
        }
    }

    // Update sidebar with topics from API
    updateSidebarTopics(topics) {
        const topicList = document.querySelector('.topic-list');
        if (topicList && topics) {
            topicList.innerHTML = topics.map(topic => `
                <li onclick="sendQuickMessage('${topic.id}')">
                    <i class="fas fa-clipboard-list"></i>
                    ${topic.title}
                </li>
            `).join('');
        }
    }
}

// Optional client-side helper (index.html contains inline client too).
// This file can be referenced by index.html if you prefer externalizing the script.

async function processMessageToServer(text) {
  if (!text || !text.trim()) return null;
  try {
    const resp = await fetch('/api/chat', {
```

```
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ message: text })
    });
    if (!resp.ok) throw new Error('Network response was not ok');
    return await resp.json();
  } catch (err) {
    return { reply: 'Error contacting server: ' + err.message, confidence: 0 };
  }
}

// Initialize enhanced chatbot
const edubot = new EnhancedEduBot();

// Global functions for HTML interactions
function sendMessage() {
    const input = document.getElementById('messageInput');
    const message = input.value.trim();

    if (message) {
        edubot.displayMessage(message, 'user');
        edubot.processMessage(message);
        input.value = '';
    }
}

function sendQuickMessage(message) {
    edubot.displayMessage(message, 'user');
    edubot.processMessage(message);
}

function handleKeyPress(event) {
    if (event.key === 'Enter') {
        sendMessage();
    }
}

function clearChat() {
    edubot.clearChat();
}

// Initialize chat on page load
document.addEventListener('DOMContentLoaded', function() {
    edubot.scrollToBottom();
    edubot.loadTopics();

    // Add some CSS for new features
    const style = document.createElement('style');
    style.textContent = `
        .suggestions {
```

```
            margin-top: 15px;
            padding: 10px;
            background: rgba(102, 126, 234, 0.05);
            border-radius: 10px;
        }

        .suggestions-label {
            font-size: 0.85rem;
            color: #666;
            margin-bottom: 8px;
            font-weight: 500;
        }

        .suggestion-buttons {
            display: flex;
            flex-wrap: wrap;
            gap: 8px;
        }

        .suggestion-btn {
            background: rgba(102, 126, 234, 0.1);
            border: 1px solid rgba(102, 126, 234, 0.3);
            color: #667eea;
            padding: 6px 12px;
            border-radius: 15px;
            font-size: 0.8rem;
            cursor: pointer;
            transition: all 0.3s ease;
        }

        .suggestion-btn:hover {
            background: #667eea;
            color: white;
            transform: translateY(-1px);
        }

        .confidence-indicator {
            margin-top: 10px;
            font-size: 0.8rem;
            color: #28a745;
            display: flex;
            align-items: center;
            gap: 5px;
        }

        .confidence-indicator i {
            font-size: 0.9rem;
        }
    `;
    document.head.appendChild(style);
```

```
371 });
```

Listing A.1: Sample Source Code of enhanced-scripts.js

References

[1] Sharma, P.; Gupta, R. (2023). Intelligent Chatbots Using Artificial Intelligence for Educational Institutions, *International Journal of Advanced Computer Science and Applications*, 13(5), 112–118.

[2] Singh, A.; Mehta, K. (2023). AI-Powered Conversational Agents for Student Query Management, *Journal of Information Technology Education: Research*, 21(2), 54–63.

[3] Kaur, S.; Verma, N. (2024). Implementation of Natural Language Processing in Academic Chatbots, *International Journal of Artificial Intelligence Research*, 11(4), 78–85.

[4] Patel, M.; Iyer, A. (2024). Machine Learning-Based Chatbot Framework for Campus Enquiry Automation, *Journal of Emerging Technologies in Computer Science*, 12(3), 210–218.

[5] Das, R.; Banerjee, P. (2022). Enhancing User Experience in Educational Chatbots Through NLP Techniques, *Journal of Computational Intelligence and Systems*, 19(1), 35–42.

[6] Raj, D.; Nair, S. (2023). A Context-Aware AI Chatbot for College Admission Assistance, *International Journal of Information Systems and Applications*, 16(2), 95–102.

[7] Thomas, L.; Joseph, R. (2024). Hyperparameter Tuning Approaches in AI Chatbots for Improved Query Accuracy, *Journal of Machine Learning and Intelligent Systems*, 10(4), 180–187.

[8] Ahmed, Z.; Khan, F. (2023). Design and Development of NLP-Based Educational Chatbots, *Journal of Computer Science and Information Technology*, 14(5), 66–74.

[9] Reddy, V.; Srinivasan, T. (2024). Integration of AI Chatbots for Enhancing Student Support Services in Universities, *International Journal of Educational Technology and Learning*, 15(2), 120–128.

[10] Banerjee, S.; Roy, D. (2023). Deep Learning Approaches for Conversational Agents in Higher Education, *Journal of Artificial Intelligence and Data Science*, 9(3), 145–153.